Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 9

Disciplina: Tehnici de programare

Tema: "Colecții: liste, tupluri, mulțimi, dicționare."

A efectuat : st. gr. SI–262
Inculeț Andrian

A verificat : asistent universitar
Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

A1. Scheletul și prima listă (15 min)

Creează arsenal/privesc/enum.py. Întâi citești fișierele SUID într-o listă și o numeri. Un fișier de citire comun îți scoate liniile curate (fără goluri, fără comentarii):

```
import argparse, json, os

def citeste_linii(cale):
    with open(cale, encoding="utf-8") as f:
        # TODO: intoarce lista liniilor .strip(), fara cele goale si fara cele cu #
        ...

cai = citeste_linii("start/suid.txt")
print("linii SUID:", len(cai))
```

Pe ecran trebuie să vezi:

```
linii SUID: 14
```

De observat. Sunt 14 rânduri, dar unele se repetă. Lista păstrează tot, în ordine, inclusiv dublurile. Le strângem la pasul următor.

A2. Nume unice: de la listă la mulțime (12 min)

Numele fișierului e ce decide dacă e periculos (find, vim, nmap...). Scoate numele din fiecare cale și ține-le o singură dată.

```
def nume(cale):
    # TODO: intoarce ce e dupa ultimul "/"  (vezi rsplit)
    ...

nume_unice = {nume(c) for c in cai}    # set comprehension
print("SUID distincte:", len(nume_unice))
```

Pe ecran trebuie să vezi:

```
SUID distincte: 13
```

De observat. 14 rânduri, 13 nume distincte: mulțimea a aruncat dublura. Dacă îți dă tot 14, verifică funcția nume: probabil întoarce toată calea, nu doar numele.

A3. Constatările SUID: listă de dicționare (18 min)

Fiecare SUID devine o fișă: calea, numele, riscul. Periculos = numele e într-o mulțime de binare cunoscute ca scăpări (stil GTFOBins). Întorci o listă de dicționare, sortată, fără dubluri.

```
GTFO = {"find", "vim.basic", "vim", "nmap", "cp", "bash",
        "less", "awk", "python3", "perl"}

def suid_constatari(cale):
    cai = citeste_linii(cale)
    # TODO: pentru fiecare cale UNICA, in ordine sortata, un dict:
    #   {"cale": c, "nume": nume(c),
    #    "risc": "mare" daca nume(c) in GTFO, altfel "normal"}
    ...

suid = suid_constatari("start/suid.txt")
mari = [c for c in suid if c["risc"] == "mare"]
```

```
print(len(suid), "SUID,", len(mari), "cu risc mare")
print(sorted(c["nume"] for c in mari))
```

Pe ecran trebuie să vezi:

```
13 SUID, 4 cu risc mare
['cp', 'find', 'nmap', 'vim.basic']
```

A4. sudo și cron: tuplul și gruparea (20 min)

O linie din sudo.txt are forma „user : drept". O despachetezi într-un tuplu. O linie din crontab.txt are șase câmpuri de program, apoi userul și comanda. Fiecare sursă dă o listă de dicționare.

```
def sudo_constatari(cale):
    out = []
    for ln in citeste_linii(cale):
        user, drept = ln.split(" : ", 1)   # despachetare in tuplu
        # TODO: adauga {"user": user.strip(), "drept": drept.strip(),
        #   "risc": "mare" daca "NOPASSWD" in drept, altfel "normal"}
        ...
    return out

def cron_constatari(cale):
    # TODO: campuri = ln.split(); user = campuri[5];
    #   comanda = " ".join(campuri[6:]); dict cu user si comanda
    ...
```

Acum le grupezi pe toate într-un singur raport: un dicționar cu o cheie pe categorie, valoarea = lista ei.

```
def enumereaza(d):
    return {
        "suid": suid_constatari(os.path.join(d, "suid.txt")),
        "sudo": sudo_constatari(os.path.join(d, "sudo.txt")),
        "cron": cron_constatari(os.path.join(d, "crontab.txt")),
    }
```

A5. Raportul: JSON și rezumat pe categorii (15 min)

main() cu argparse: primește folderul capturilor și un comutator --json. Cu --json scoate JSON curat; fără el, un rezumat pe categorii (cu .items()) și salvează raportul pe disc.

```
def main():
    p = argparse.ArgumentParser()
    p.add_argument("capturi")
    p.add_argument("--json", action="store_true")
    a = p.parse_args()
    raport = enumereaza(a.capturi)
    if a.json:
        print(json.dumps(raport, indent=2, ensure_ascii=False)); return
    for categorie, lista in raport.items():
        mari = [c for c in lista if c.get("risc") == "mare"]
        print(f"{categorie}: {len(lista)} gasite, {len(mari)} cu risc mare")
    # TODO: utilizatorii implicati = multimea userilor din sudo REUNITA cu cei din cron
    with open("enum.json", "w", encoding="utf-8") as f:
        json.dump(raport, f, indent=2, ensure_ascii=False)
    print("Raport salvat in enum.json")
```

```
if __name__ == "__main__":
    main()
```

Rulezi din rădăcina depozitului, cu -m, dând calea capturilor:

```
python -m arsenal.privesc.enum arsenal/privesc/start
```

Pe ecran trebuie să vezi:

```
suid: 13 gasite, 4 cu risc mare
sudo: 3 gasite, 2 cu risc mare
cron: 3 gasite, 0 cu risc mare
utilizatori implicati: ana, root, www-data
Raport salvat in enum.json
```

DE REȚINUT. Patru colecții, fiecare la locul ei: lista ține fișierele în ordine, tuplul despachetează o linie, mulțimea scoate userii unici, dicționarul grupează raportul. Codul a ieșit scurt pentru că structura a fost aleasă bine.

```
import argparse, json, os

def citeste_linii(cale):
    with open(cale, encoding="utf-8") as f:
        linii = f.read().split("\n")
        return [linie.strip() for linie in linii if linie and not
linie[0].startswith("#")]

cai = citeste_linii("./arsenal/privesc/start/suid.txt")


def nume(cale):
    return cale.rsplit("/")[-1]


nume_unice = {nume(c) for c in cai}  # set comprehension


GTFO = {"find", "vim.basic", "vim", "nmap", "cp", "bash",
        "less", "awk", "python3", "perl"}


def suid_constatari(cale):
    cai = citeste_linii(cale)

    cai_unice_sortate = sorted(set(cai))

    rezultat = []

    for cale in cai_unice_sortate:
        nume_bin = cale.rsplit("/")[-1]

        risc = "mare" if nume_bin in GTFO else "normal"
        rezultat.append({
            "cale": cale,
            "nume": nume_bin,
```

```
            "risc": risc,
        })

    return rezultat


suid = suid_constatari("./arsenal/privesc/start/suid.txt")
mari = [c for c in suid if c["risc"] == "mare"]

def sudo_constatari(cale):
    out = []
    for ln in citeste_linii(cale):
        user, drept = ln.split(" : ", 1)  # despachetare in tuplu

        risc = "mare" if "NOPASSWD" in drept else "normal"

        out.append({
            "user": user.strip(),
            "drept": drept.strip(),
            "risc": risc
        })
    return out


def cron_constatari(cale):
    out = []

    for linie in citeste_linii(cale):
        campuri = linie.split()

        if len(campuri) < 7:
            continue

        user = campuri[5]
        comanda = " ".join(campuri[6:])



        out.append({
            "user": user,
            "comanda": comanda,
        })

    return out


def enumereaza(d):
    return {
        "suid": suid_constatari(os.path.join(d, "suid.txt")),
        "sudo": sudo_constatari(os.path.join(d, "sudo.txt")),
        "cron": cron_constatari(os.path.join(d, "crontab.txt")),
    }

def main():
    p = argparse.ArgumentParser()
    p.add_argument("capturi")
    p.add_argument("--json", action="store_true")
```

```
    a = p.parse_args()

    raport = enumereaza(a.capturi)

    if a.risc_mare:
        raport = {
            categorie: [c for c in lista if c.get("risc") == "mare"] for categorie,
lista in raport.items()
        }

    if a.json:
        print(json.dumps(raport, indent=2, ensure_ascii=False));
        return

    for categorie, lista in raport.items():
        mari = [c for c in lista if c.get("risc") == "mare"]
        print(f"{categorie}: {len(lista)} gasite, {len(mari)} cu risc mare")

    utilizatori_sudo = {c["user"] for c in raport["sudo"]}
    utilizatori_cron = {c["user"] for c in raport["cron"]}
    useri_implicati = utilizatori_cron.union(utilizatori_sudo)

    print("Utilizatori implicati:", ", ".join(sorted(useri_implicati)))

    # TODO: utilizatorii implicati = multimea userilor din sudo REUNITA cu cei din
cron
    with open("./arsenal/privesc/enum.json", "w", encoding="utf-8") as f:
        json.dump(raport, f, indent=2, ensure_ascii=False)
    print("Raport salvat in enum.json")



if __name__ == "__main__":

    print("A1.")
    cai = citeste_linii("./arsenal/privesc/start/suid.txt")
    print("linii SUID:", len(cai))

    print("\n\nA2.")
    nume_unice = {nume(c) for c in cai}
    print("SUID distincte:", len(nume_unice))

    print("\n\nA3.")
    suid = suid_constatari("./arsenal/privesc/start/suid.txt")
    mari = [c for c in suid if c["risc"] == "mare"]
    print(len(suid), "SUID,", len(mari), "cu risc mare")
    print(sorted(c["nume"] for c in mari))

    print("\n\nA4. (Functii definite)")

    print("\n\nA5.")

    main()
```

Afisare:

```
(.venv) xperiencez@lenovo-v15-g4-iru:~/Documents/GitHub/tehnici-de-programare$ python
```

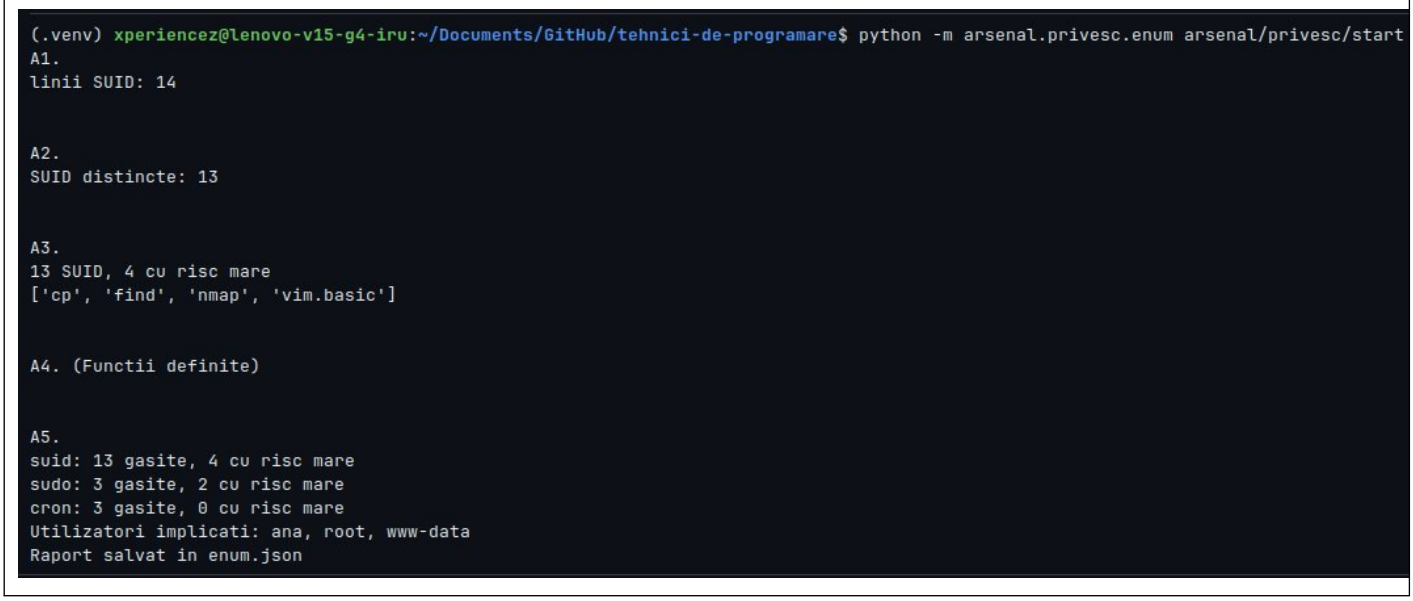
```
-m arsenal.privesc.enum arsenal/privesc/start
A1.
linii SUID: 14


A2.
SUID distincte: 13


A3.
13 SUID, 4 cu risc mare
['cp', 'find', 'nmap', 'vim.basic']


A4. (Functii definite)


A5.
suid: 13 gasite, 4 cu risc mare
sudo: 3 gasite, 2 cu risc mare
cron: 3 gasite, 0 cu risc mare
Utilizatori implicati: ana, root, www-data
Raport salvat in enum.json
```



enum.py


```
{
  "suid": [
    {
      "cale": "/bin/cp",
      "nume": "cp",
      "risc": "mare"
    },
    {
      "cale": "/bin/mount",
      "nume": "mount",
```

```
      "risc": "normal"
    },
    {
      "cale": "/bin/su",
      "nume": "su",
      "risc": "normal"
    },
    {
      "cale": "/bin/umount",
      "nume": "umount",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/chfn",
      "nume": "chfn",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/chsh",
      "nume": "chsh",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/find",
      "nume": "find",
      "risc": "mare"
    },
    {
      "cale": "/usr/bin/gpasswd",
      "nume": "gpasswd",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/newgrp",
      "nume": "newgrp",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/nmap",
      "nume": "nmap",
      "risc": "mare"
    },
    {
      "cale": "/usr/bin/passwd",
      "nume": "passwd",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/sudo",
      "nume": "sudo",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/vim.basic",
      "nume": "vim.basic",
      "risc": "mare"
    }
  ],
  "sudo": [
```

```
    {
      "user": "ana",
      "drept": "(ALL) NOPASSWD: /usr/bin/vim.basic",
      "risc": "mare"
    },
    {
      "user": "ana",
      "drept": "(ALL) /usr/bin/apt",
      "risc": "normal"
    },
    {
      "user": "www-data",
      "drept": "(root) NOPASSWD: /usr/bin/find",
      "risc": "mare"
    }
  ],
  "cron": [
    {
      "user": "root",
      "comanda": "cd / && run-parts --report /etc/cron.hourly"
    },
    {
      "user": "root",
      "comanda": "/opt/backup/run.sh"
    },
    {
      "user": "www-data",
      "comanda": "/home/www-data/clean.sh"
    }
  ]
}
```

B1. Două mulțimi: curat și acum (15 min)

```
def cai(fisier):
    # TODO: intoarce MULTIMEA cailor din fisier (foloseste citeste_linii din enum)
    ...

baza = cai("start/baza_suid.txt")
acum = cai("start/suid.txt")
print("baza:", len(baza), " acum:", len(acum))
```

Pe ecran trebuie să vezi:

```
baza: 9  acum: 13
```

B2. Diferența: ce a apărut (15 min)

Ce e în „acum" dar nu în „bază" sunt fișierele SUID noi. Un singur operator, nu o buclă dublă.

```
noi = acum - baza
print(len(noi), "SUID noi fata de sistemul curat")
for c in sorted(noi):
    print(" ", c)
```

Pe ecran trebuie să vezi:

```
4 SUID noi fata de sistemul curat
```

```
/bin/cp
/usr/bin/find
/usr/bin/nmap
/usr/bin/vim.basic
```

B3. Dicționar de risc și filtrul final (20 min)

Faci o fișă nume -> risc pentru cele noi (dict comprehension), apoi filtrezi doar periculoasele (list comprehension cu if). Refolosești nume() și GTFO din enum printr-un import.

```
from arsenal.privesc.enum import nume, GTFO

risc = {nume(c): ("mare" if nume(c) in GTFO else "normal") for c in noi}
periculoase = [c for c in sorted(noi) if nume(c) in GTFO]
print("periculoase NOI:", periculoase)
```

Pe ecran trebuie să vezi:

```
periculoase NOI: ['/bin/cp', '/usr/bin/find', '/usr/bin/nmap', '/usr/bin/vim.basic']
```

De observat. Toate cele patru apărute sunt periculoase: cineva a pus bit SUID pe find, nmap, cp și vim. Oricare dă root. Asta e exact genul de constatare pe care o scoți în față, nu o îngropi în raport.

B4. Un test care prinde regresia (15 min)

Scrie arsenal/privesc/test_diferenta.py cu un test mic, cu date scrise de mână, care verifică faptul cheie: ce e în acum și nu în bază iese corect. Rulează cu pytest și treacă fără rețea.

```
def test_doar_ce_e_nou():
    acum = {"a", "b", "c"}
    baza = {"a", "b"}
    assert acum - baza == {"c"}

pytest -q arsenal/privesc/test_diferenta.py
```

Pe ecran trebuie să vezi:

```
1 passed
```

```
from arsenal.privesc.enum import citeste_linii, nume, GTFO


def cai(fisier):
    linii = citeste_linii(fisier)
    return set(linii)

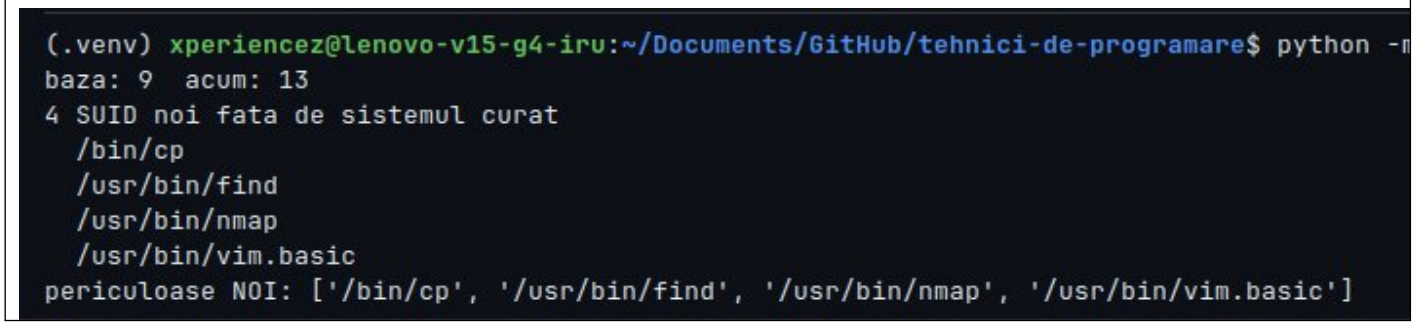

baza = cai("./arsenal/privesc/start/baza_suid.txt")
acum = cai("./arsenal/privesc/start/suid.txt")
print("baza:", len(baza), " acum:", len(acum))


noi = acum - baza
print(len(noi), "SUID noi fata de sistemul curat")
for c in sorted(noi):
    print(" ", c)
```

```
risc = {nume(c): ("mare" if nume(c) in GTFO else "normal") for c in noi}
periculoase = [c for c in sorted(noi) if nume(c) in GTFO]
print("periculoase NOI:", periculoase)
```

Output:

```
baza: 9  acum: 13
4 SUID noi fata de sistemul curat
  /bin/cp
  /usr/bin/find
  /usr/bin/nmap
  /usr/bin/vim.basic
periculoase NOI: ['/bin/cp', '/usr/bin/find', '/usr/bin/nmap', '/usr/bin/vim.basic']
```



pytest

```
def test_doar_ce_e_nou():
    acum = {"a", "b", "c"}
    baza = {"a", "b"}
    assert acum - baza == {"c"}
```

Afisare:

```
(.venv) xperiencez@lenovo-v15-g4-iru:~/Documents/GitHub/tehnici-de-programare$ pytest
-q arsenal/privesc/test_diferenta.py
.
[100%]
1 passed in 0.00s
```



OPȚIONAL (pentru vârful sălii sau acasă)

1. --risc mare: un comutator care scoate din raport doar constatările cu risc mare, din toate categoriile, printr-un comprehension.
2. enum.py pe o țintă reală: înlocuiește citirea din fișier cu subprocess.run(["find", "/", "-perm", "-4000"], ...) pe mașina Kali/Metasploitable din laborator, izolată. Compari cu capturile date.
3. Grupare după risc: un dicționar risc -> listă de nume, construit cu .setdefault, din toate categoriile deodată

Am facut 1 si 3

```
def main():
```

```
    p = argparse.ArgumentParser()
    p.add_argument("capturi")
    p.add_argument("--json", action="store_true")
    p.add_argument("--risc-mare", action="store_true")
    a = p.parse_args()

    raport = enumereaza(a.capturi)

    if a.risc_mare:
        raport = {
            categorie: [c for c in lista if c.get("risc") == "mare"] for categorie,
lista in raport.items()
        }

    if a.json:
        print(json.dumps(raport, indent=2, ensure_ascii=False));
        return

    for categorie, lista in raport.items():
        mari = [c for c in lista if c.get("risc") == "mare"]
        print(f"{categorie}: {len(lista)} gasite, {len(mari)} cu risc mare")

    utilizatori_sudo = {c["user"] for c in raport["sudo"]}
    utilizatori_cron = {c["user"] for c in raport["cron"]}
    useri_implicati = utilizatori_cron.union(utilizatori_sudo)

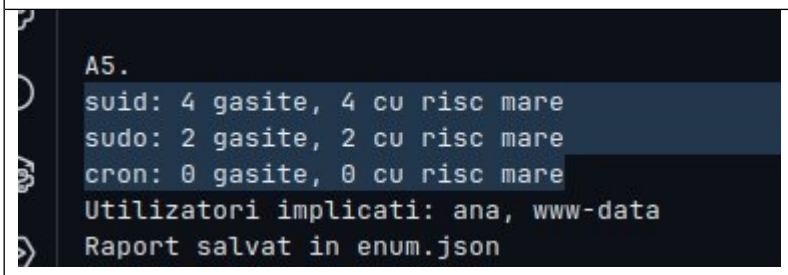
    print("Utilizatori implicati:", ", ".join(sorted(useri_implicati)))

    # TODO: utilizatorii implicati = multimea userilor din sudo REUNITA cu cei din
cron
    with open("./arsenal/privesc/enum.json", "w", encoding="utf-8") as f:
        json.dump(raport, f, indent=2, ensure_ascii=False)
    print("Raport salvat in enum.json")


    grupuri = grupeaza_dupa_risc(raport)
    print("\n\nGrupare dupa risc:")

    for risc, nume_binare in grupuri.items():
        print(f"{risc} : {nume_binare}")
```

Afisare, cu python -m arsenal.privesc.enum arsenal/privesc/start --risc-mare



Afisare cu python -m arsenal.privesc.enum arsenal/privesc/start

